Guía Completa de Bucles en JavaScript

Estructuras de control, métodos de array funcionales y sintaxis de iteración moderna

1. BUCLES TRADICIONALES DE CONTROL

1. Bucle `for` Estándar

Control / Contador

Ideal cuando conoces de antemano el número exacto de iteraciones o necesitas un contador numérico explícito.

ESTRUCTURA

```
for (inicializacion; condicion; incremento) {
  // Código a ejecutar
}
```

EJEMPLO

```
for (let i = 0; i < 3; i++) {
  console.log(`Índice: ${i}`);
}
```

2. Bucle `while`

Condicional previo

Ejecuta el bloque mientras se cumpla la condición. Evalúa la condición **antes** de ejecutar el bloque.

ESTRUCTURA

```
while (condicion) {
  // Código a ejecutar
  // Actualización de variable
}
```

EJEMPLO

```
let n = 0;
while (n < 3) {
  console.log(n);
  n++;
}
```

3. Bucle `do...while`

Condicional posterior

Garantiza la ejecución del bloque **al menos una vez**, ya que la condición se evalúa al final.

ESTRUCTURA

```
do {
  // Código (se ejecuta al menos 1 vez)
} while (condicion);
```

EJEMPLO

```
let x = 5;
do {
  console.log(x); // Imprime 5
  x++;
} while (x < 3);
```

2. ITERACIÓN DE COLECCIONES Y OBJETOS

4. Bucle `for...of`

Valores de Iterables

Recorre directamente los **valores** de objetos iterables (Arrays, Strings, Maps, Sets, NodeLists).

ESTRUCTURA

```
for (const elemento of iterable) {
  // Operación con el elemento
}
```

EJEMPLO

```
const frutas = ["manzana", "pera"];
for (const fruta of frutas) {
  console.log(fruta);
}
```

5. Bucle `for...in`

Propiedades de Objetos

Itera sobre las **claves/propiedades enumerables** de un objeto.

ESTRUCTURA

```
for (const clave in objeto) {
  // Acceso con objeto[clave]
}
```

EJEMPLO

```
const user = { nombre: "Ana", rol: "dev" };
for (const k in user) {
  console.log(`${k}: ${user[k]}`);
}
```

6. Bucle `for await...of`

Iteración Asíncrona

Itera sobre colecciones que entregan **Promesas** o colecciones asíncronas de datos.

ESTRUCTURA

```
for await (const item of asyncIterable) {
  // Espera la resolución de cada promesa
}
```

EJEMPLO

```
async function procesar(promesas) {
  for await (const res of promesas) {
    console.log(res);
  }
}
```

3. MÉTODOS ITERATIVOS DE ARRAY Y FUNCIONES FLECHA

7. Método `forEach()`

Efectos Secundarios

Ejecuta un callback para cada elemento. No devuelve nada (`undefined`).

ESTRUCTURA

```
arr.forEach((elem, idx) => {
  // Acción por elemento
});
```

EJEMPLO

```
[10, 20].forEach((num, i) => {
  console.log(`[${i}]: ${num}`);
});
```

8. Método `map()`

Transformación de Datos

Transforma cada elemento y **devuelve un nuevo Array** con los resultados.

ESTRUCTURA

```
const nuevoArr = arr.map(elem => {
  return valorTransformado;
});
```

EJEMPLO

```
const nums = [1, 2, 3];
const dobles = nums.map(n => n * 2);
// dobles es [2, 4, 6]
```

9. Funciones Flecha (Arrow Functions en Iteraciones)

Sintaxis Callback

Sintaxis concisa para escribir callbacks en métodos de iteración. Soporta retorno implícito.

ESTRUCTURA

```
// Retorno implícito (1 línea)
param => expresion

// Bloque con return explícito
(p1, p2) => { return resultado; }
```

EJEMPLO

```
const nombres = ["ana", "juan"];
const mayus = nombres.map(n => n.toUpperCase
());
// ["ANA", "JUAN"]
```

4. CONTROL AVANZADO CON ETIQUETAS

10. Bucles Etiquetados (`label: for`)

Control Anidado

Permite asignar un nombre a un bucle externo para interrumpirlo con `break` o `continue` desde un bucle interno.

ESTRUCTURA

```
etiqueta: for (...) {
  for (...) {
    if (cond) break etiqueta;
  }
}
```

EJEMPLO

```
ext: for (let i = 0; i < 2; i++) {
  for (let j = 0; j < 2; j++) {
    if (i === 1) break ext;
  }
}
```